

Synchronization Examples

Module 4

Classical Problems of Synchronization

Classical problems used to test newly-proposed synchronization schemes

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

Classic Problems of Synchronization

(The Bounded-Buffer Problem)

The Bounded Buffer Problem (**Producer Consumer Problem**), is one of the classic problems of synchronization.

There is a buffer of n slots and each slot is capable of storing one unit of data.

There are two processes running, namely, **Producer** and **Consumer**, which are operating on the buffer.



Bounded-Buffer Problem

- The producer tries to insert data into an empty slot of the buffer.
- The consumer tries to remove data from a filled slot in the buffer.
- The Producer must not insert data when the buffer is full.
- The Consumer must not remove data when the buffer is empty.
- The Producer and Consumer should not insert and remove data simultaneously.

- n buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n

1. m (mutex), a binary semaphore which is used to acquire and release the lock.
2. empty, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
3. full, a counting semaphore whose initial value is 0.

Producer

```
do {  
    wait (empty); // wait until empty>0  
                  and then decrement 'empty'  
    wait (mutex); // acquire lock  
    /* add data to buffer */  
    signal (mutex); // release lock  
    signal (full); // increment 'full'  
} while(TRUE)
```

Consumer

```
do {  
    wait (full); // wait until full>0 and  
                then decrement 'full'  
    wait (mutex); // acquire lock  
    /* remove data from buffer */  
    signal (mutex); // release lock  
    signal (empty); // increment 'empty'  
} while(TRUE)
```

Bounded-Buffer Problem

- The structure of the producer process

```
while (true) {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
}
```

Bounded-Buffer Problem

- The structure of the consumer process

```
while (true) {  
    wait(full);  
    wait(mutex);  
  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
  
    /* consume the item in next consumed */  
    ...  
}
```


Readers-Writers Problem

Classic Problems of Synchronization (The Readers-Writers Problem)

- A database is to be shared among several concurrent processes.
- Some of these processes may want **only to read the database**, whereas **others may want to update (that is, to read and write) the database**.
- We distinguish between these two types of processes by referring to the former as **Readers** and to the latter as **Writers**.
- Obviously, if two readers access the shared data simultaneously, no adverse affects will result.
- However, if a writer and some other thread (either a reader or a writer) access the database simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database.

This synchronization problem is referred to as the readers-writers problem.

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do *not* perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

Readers-Writers Problem

- Shared Data
 - Data set
 - Semaphore `rw_mutex` initialized to 1
 - Semaphore `mutex` initialized to 1
 - Integer `read_count` initialized to 0

Solution to the Readers-Writers Problem using Semaphores:

We will make use of **two semaphores** and an **integer variable**:

1. **`mutex`**, a semaphore (**initialized to 1**) which is used to ensure mutual exclusion when **`readcount`** is updated i.e. when any reader enters or exit from the critical section.
2. **`wrt`**, a semaphore (**initialized to 1**) common to both reader and writer processes.
3. **`readcount`**, an integer variable (**initialized to 0**) that keeps track of how many processes are currently reading the object. 

Readers-Writers Problem

Writer Process	Reader Process
<pre>do { /* writer requests for critical section */ wait(wrt); /* performs the write */ // leaves the critical section signal(wrt); } while(true);</pre>	<pre>do { wait (mutex); readcnt++; // The number of readers has now increased by 1 if (readcnt==1) wait (wrt); // this ensure no writer can enter if there is even one reader signal (mutex); // other readers can enter while this current reader is inside the critical section /* current reader performs reading here */ wait (mutex); readcnt--; // a reader wants to leave if (readcnt == 0) //no reader is left in the critical section signal (wrt); // writers can enter signal (mutex); // reader leaves } while(true);</pre>

Readers-Writers Problem

- The structure of a writer process

```
while (true) {  
    wait(rw_mutex) ;  
  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex) ;  
}
```

Readers-Writers Problem

- The structure of a reader process

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1) /* first reader */
        wait(rw_mutex);
    signal(mutex);

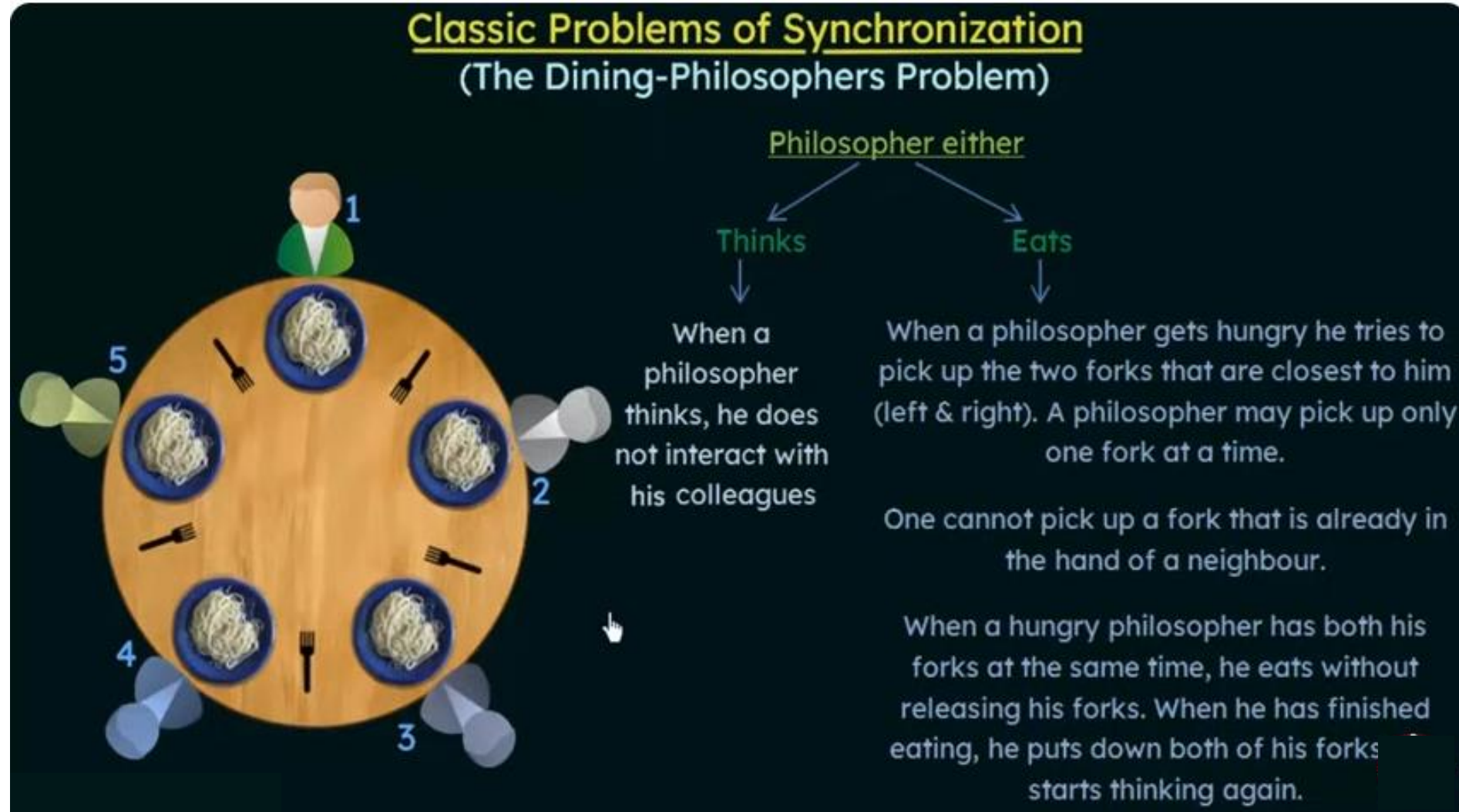
    ... /* reading is performed */

    ...
    wait(mutex);
    read_count--;
    if (read_count == 0) /* last reader */
        signal(rw_mutex);
    signal(mutex);
}
```

Readers-Writers Problem Variations

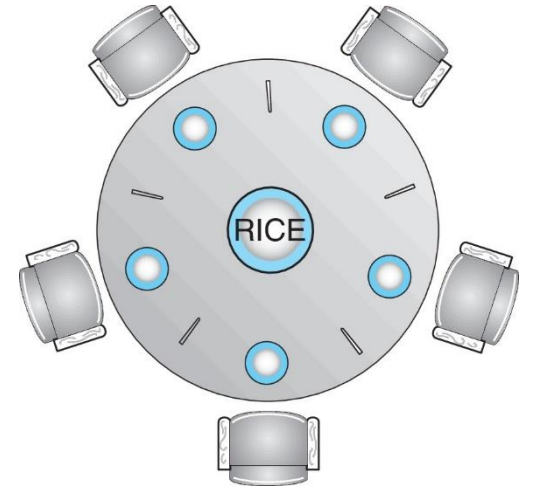
- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
 - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation. leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

Dining-Philosophers Problem



Dining-Philosophers Problem

- N philosophers' sit at a round table with a bowl of rice in the middle.
- They spend their lives alternating thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
 - Bowl of rice (data set)
 - Semaphore chopstick [5] initialized to 1



Dining-Philosophers Problem

One simple solution is to represent each fork/chopstick with a **semaphore**.

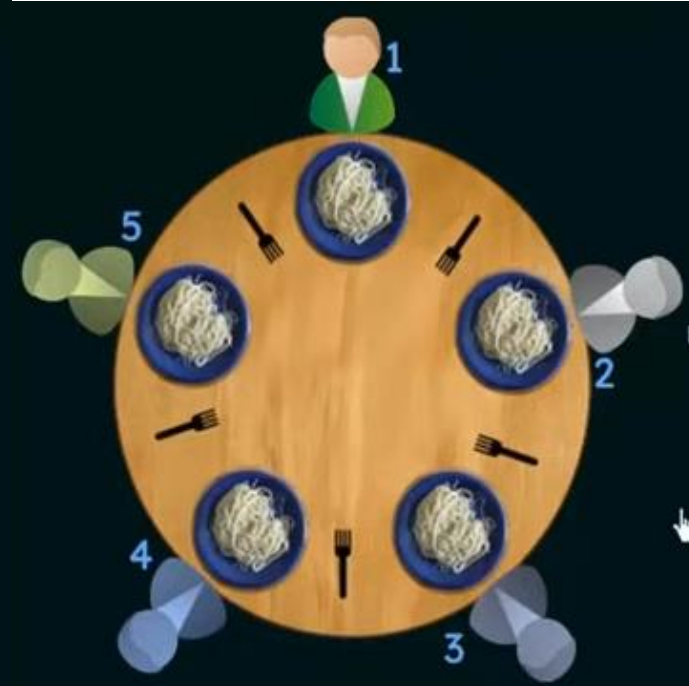
A philosopher tries to **grab a fork/chopstick** by executing a **wait ()** operation on that semaphore.

He **releases his fork/chopsticks** by executing the **signal ()** operation on the appropriate semaphores.

Thus, the shared data are

```
semaphore chopstick[5];
```

where all the elements of chopstick are initialized to **1**.



Dining-Philosophers Problem Algorithm

Semaphore Solution

- The structure of Philosopher i :

```
while (true){  
    wait (chopstick[i] );  
    wait (chopStick[ (i + 1) % 5] );  
  
    /* eat for awhile */  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    /* think for awhile */  
  
}
```

Dining-Philosophers Problem Algorithm

The structure of philosopher i

```
do {  
    wait (chopstick [i] ) ;  
    wait(chopstick [ (i + 1) % 5] ) ;  
    ....  
    // eat  
    signal(chopstick [i]);  
    signal(chopstick [(i + 1) % 5]);  
    // think  
}while (TRUE);
```

Although this solution guarantees that no two neighbors are eating simultaneously, it could still create a deadlock.

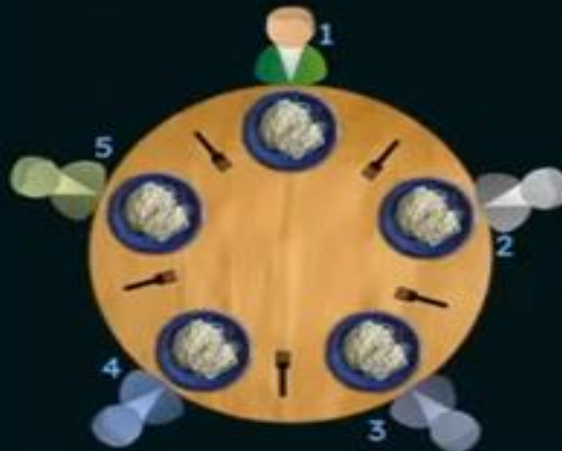
Suppose that all five philosophers become hungry simultaneously and each grabs their left chopstick. All the elements of chopstick will now be equal to 0.

When each philosopher tries to grab his right chopstick, he will be delayed forever.

Solution to Dining Philosophers

Some possible remedies to avoid deadlocks:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this he must pick them up in a critical section).
- Use an asymmetric solution; that is, an odd philosopher picks up first his left chopstick and then his right chopstick, whereas an even philosopher picks up his right chopstick and then his left chopstick.



Dining-Philosophers Solution Using Monitors

- We now illustrate monitor concepts by presenting a deadlock-free solution to the dining-philosophers problem.
- This solution imposes the restriction that a philosopher may pick up his chopsticks only if both of them are available.
- To code this solution, we need to distinguish among three states in which we may find a philosopher. For this purpose, we introduce the following data structure:

```
enum { thinking, hungry, eating } state [5];
```

- Philosopher i can set the variable $state[i] = eating$ only if his two neighbors are not eating: $(state[(i+4) \% 5] \neq eating)$ and $(state[(i+1)\% 5] \neq eating)$.
- We also need to declare `condition self [5];` where philosopher i can delay himself when he is hungry but is unable to obtain the chopsticks he needs.

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
```

```
{  
    enum {THINKING; HUNGRY, EATING} state [5];  
    condition self [5];  
  
    void pickup (int i) {  
        state[i] = HUNGRY;  
        test(i);  
        if (state[i] != EATING) self[i].wait;  
    }  
    void putdown (int i) {  
        state[i] = THINKING;  
        // test left and right neighbors  
        test((i + 4) % 5);  
        test((i + 1) % 5);  
    }  
}
```

Solution to Dining Philosophers (Cont.)

```
void test (int i) {  
    if ((state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}
```

```
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```

```
}
```

```
monitor dp {  
    enum { THINKING, HUNGRY, EATING } state [5];  
    condition self [5];
```

```
    void pickup (int i) {  
        state [i] = HUNGRY;  
        test (i);  
        if (state [i] != EATING)  
            self [i].wait();  
    }
```

```
    void putdown(int i) {  
        state [i] = THINKING;  
        test ((i + 4) % 5);  
        test ((i + 1) % 5);  
    }
```

```
    void test (int i) {  
        if ((state [(i + 4) % 5] != EATING) &&  
            (state [i] == HUNGRY) &&  
            (state [(i + 1) % 5] != EATING)) {  
            state [i] = EATING;  
            self [i].signal();  
        }  
    }
```

```
    initialization-code () {  
        for (int i = 0; i < 5; i++)  
            state [i] = THINKING;  
    }
```

```
}
```

Solution to Dining Philosophers

- Each philosopher “i” invokes the operations **pickup()** and **putdown()** in the following sequence:

```
DiningPhilosophers.pickup(i);
```

```
/** EAT **/
```

```
DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible